

AI-Powered Resume Analyzer & Ontology-Based Job Recommender

Complete Step-by-Step Build Guide

Project Overview

This guide walks you through building a full-stack AI-powered web application that:

- Parses a user-uploaded resume (PDF/DOCX) to extract skills, experience, and education.
- Compares the candidate's profile against an Ontology-based skill graph stored in Neo4j to generate a Skill Gap Analysis for a chosen target job role.
- Recommends the Top 5–10 jobs the candidate is most likely to get hired for, based on resume-to-job similarity scoring.
- Pulls job listings daily (via scraping or APIs), enriches them with skill metadata, and stores them in Neo4j for matching.

Your Architecture Instinct Is Correct

Your idea of using Neo4j for an ontology-based skills graph is spot-on. Neo4j excels at modeling relationships between Skills, Job Roles, Candidates, and Concepts — exactly the kind of connected data this project needs.

Recommended Technology Stack

Component	Technology	Why / Notes
Frontend	React.js + Tailwind CSS	Upload UI, results dashboard
Backend API	Python + FastAPI	Fast, async, great ML ecosystem
Resume Parsing	PyMuPDF / pdfplumber + spaCy	Extract text, named entities

Component	Technology	Why / Notes
NLP / Skill Extraction	spaCy + BERT / Sentence Transformers	Semantic skill matching
Graph Database	Neo4j (AuraDB or local)	Skills ontology + job graph
Job Scraping	Scrapy or Playwright + APScheduler	Daily job fetch with scheduling
Relational DB	PostgreSQL	User accounts, resume history
Caching / Queue	Redis + Celery	Background job processing
Embeddings / AI	OpenAI API or Hugging Face	Semantic similarity scoring
Deployment	Docker + Docker Compose	Containerized services

Phase 1: Project Setup & Architecture

Step 1.1 — Define Your Folder Structure

Create a monorepo with clear separation between frontend, backend, and data services:

Recommended Project Structure

```
/resume-analyzer /frontend    → React app /backend    → FastAPI server /scraper    → Job  
scraping service /ontology    → Neo4j seed scripts & Cypher queries /ml    → NLP models &  
embedding scripts docker-compose.yml → Spins up all services
```

Step 1.2 — Spin Up Core Infrastructure with Docker

Create a docker-compose.yml that includes these services running together:

- FastAPI backend on port 8000
- React frontend on port 3000
- Neo4j on port 7687 (Bolt) + 7474 (browser UI)
- PostgreSQL on port 5432
- Redis on port 6379

Phase 2: Building the Skills Ontology in Neo4j

This is the most critical and foundational step. The ontology only needs to be built once and then maintained/extended over time.

Step 2.1 — Design the Graph Data Model

Your Neo4j graph will have these node types and relationships:

Node Types

```
(:Skill {name, category, aliases}) (:JobRole {title, seniority}) (:Industry {name}) (:Certification {name, issuer})  
(:Tool {name, vendor})
```

Relationship Types

```
(:JobRole)-[:REQUIRES_SKILL {weight}]->(:Skill) (:Skill)-[:RELATED_TO {similarity}]->(:Skill) (:Skill)-[:SUBSET_OF]-  
>(:Skill) // e.g., React IS_SUBSET_OF JavaScript (:JobRole)-[:BELONGS_TO]->(:Industry) (:Certification)-  
[:VALIDATES]->(:Skill)
```

Step 2.2 — Seed the Ontology with Initial Data

You have two good approaches for building the initial ontology:

- Use ESCO (European Skills, Competences, Qualifications) — a free, structured EU skill taxonomy with thousands of skills and job relationships. Download the CSV/JSON and import into Neo4j using a Python script.
- Use O*NET (US Department of Labor) — another free taxonomy mapping 1000+ occupations to required skills, tools, and knowledge areas.
- Augment with your own scraped job data (Phase 3) to add real-world skill frequency data.

Example Cypher to create a node and relationship:

Sample Cypher Query

```
MERGE (s:Skill {name: "Python"}) MERGE (j:JobRole {title: "Data Scientist"}) MERGE (j)-[:REQUIRES_SKILL  
{weight: 0.95, is_must_have: true}]->(s)
```

Step 2.3 — Add Skill Similarity Edges

Use a Sentence Transformer model (e.g., all-MiniLM-L6-v2 from Hugging Face) to compute cosine similarity between skills. Add a RELATED_TO edge between skills with similarity above a threshold (e.g., 0.7). This allows the system to match 'Machine Learning' on a resume to a job requiring 'Deep Learning' with partial credit.

Phase 3: Daily Job Scraping & Enrichment Pipeline

This service runs on a schedule (e.g., every day at 2 AM) and keeps your job database fresh.

Step 3.1 — Set Up the Scraper

You have two options — choose based on your legal/compliance needs:

- **Official Job APIs (Recommended):** LinkedIn Jobs API, Indeed Publisher API, Adzuna API, The Muse API. These are legal, structured, and provide clean data.
- **Web Scraping (Fallback):** Use Scrapy or Playwright to scrape public job boards. Always check the site's robots.txt and Terms of Service before scraping.

Step 3.2 — Extract Skills from Job Descriptions (NLP)

For each job posting fetched, run it through a Named Entity Recognition (NER) pipeline to extract skills mentioned in the description:

1. Load job description text into spaCy.
2. Use a custom-trained NER model or a matcher pattern list to tag skill entities (e.g., 'Python', 'AWS', 'Agile').
3. Match extracted skills against your Neo4j skill nodes (exact match + fuzzy/semantic fallback).
4. Store the job in Neo4j with edges to all matched skills.

Neo4j Job Node Structure

```
(:Job {id, title, company, location, apply_url, posted_date, description}) (:Job)-[:REQUIRES_SKILL {frequency}]->(:Skill) (:Job)-[:IS_ROLE]->(:JobRole)
```

Step 3.3 — Schedule with APScheduler + Celery

Wrap the scrape-and-enrich pipeline as a Celery background task. Use APScheduler or a simple cron trigger to kick it off daily. This way your FastAPI server stays responsive while scraping happens in the background.

Phase 4: Resume Parsing Service

Step 4.1 — Build the Resume Upload Endpoint

Create a POST /api/upload-resume endpoint in FastAPI that:

5. Accepts a PDF or DOCX file upload.
6. Saves the file temporarily and extracts raw text using pdfplumber (for PDFs) or python-docx (for DOCX files).
7. Returns a structured JSON with candidate name, email, phone, skills, work experience, and education sections.

Step 4.2 — Extract Skills from Resume (NER)

Use the same NLP pipeline built in Phase 3 to extract skill mentions from the resume text. Additionally:

- Map skills to canonical Neo4j skill nodes (handle synonyms — e.g., 'JS' maps to 'JavaScript').
- Infer implicit skills from job titles listed (e.g., a resume showing 'Data Analyst at Google' implies SQL, Excel, data visualization).
- Assign confidence scores to each extracted skill (high confidence for explicit mentions, lower for inferred).

Step 4.3 — Build a Candidate Profile Vector

Generate an embedding vector for the candidate's full profile by averaging the sentence transformer embeddings of their skills, job titles, and summary. This vector will be used for similarity matching against jobs in Phase 5.

Phase 5: Skill Gap Analysis

Step 5.1 — Query Neo4j for Target Job Role Requirements

When the user selects a Target Job Role (e.g., 'Senior Data Scientist'), query Neo4j:

Skill Gap Cypher Query

```
MATCH (j:JobRole {title: $target_role})-[r:REQUIRES_SKILL]->(s:Skill) RETURN s.name AS skill, r.weight AS importance, r.is_must_have AS required ORDER BY r.weight DESC
```

Step 5.2 — Compare Candidate Skills vs Required Skills

With the candidate's extracted skills and the job role's required skills in hand:

8. Exact matches: Skills present in both lists — mark as 'Have'.
9. Semantic matches: Use RELATED_TO edges in Neo4j or cosine similarity to catch partial matches — mark as 'Partial'.
10. Gaps: Required skills not in candidate's profile — mark as 'Missing'.
11. Bonus skills: Candidate has skills not required — mark as 'Extra' (could be useful for other roles).

Step 5.3 — Generate the Gap Report

Return a structured response to the frontend with:

- Match Score (e.g., 72%) — percentage of required skills covered.
- Must-Have Gaps — critical missing skills that block hiring.
- Nice-to-Have Gaps — secondary skills to improve candidacy.
- Learning Recommendations — for each missing skill, suggest a course or resource (can hardcode a curated list or call a Udemy/Coursera API).

Phase 6: Top Job Recommendations Engine

Step 6.1 — Score Each Job Against the Candidate Profile

This is a two-layer scoring approach — graph-based + vector-based:

Layer 1: Graph-Based Skill Overlap Score (Neo4j Cypher)

Job Matching Query

```
MATCH (j:Job)-[:REQUIRES_SKILL]->(s:Skill) WHERE s.name IN $candidate_skills WITH j, COUNT(s) AS matched_skills, j.total_required_skills AS total RETURN j.title, j.company, j.apply_url, (matched_skills * 1.0 / total) AS match_ratio ORDER BY match_ratio DESC LIMIT 20
```

Layer 2: Semantic Vector Similarity Score

Take the top 20 graph candidates, then re-rank them using cosine similarity between the candidate's profile embedding and each job's description embedding (pre-computed and stored when jobs were scraped). This catches cases where a candidate's skills match semantically but not by exact keyword.

Step 6.2 — Final Ranking & Why-Score Explanation

Combine both scores (e.g., 60% graph score + 40% semantic score) to produce a final ranked list. For each recommended job, generate a human-readable explanation:

- 'You match 8 of 10 required skills for this role.'
- 'Your Python and SQL skills are a strong fit.'
- 'You are missing: Docker, Kubernetes (nice to have).'

Phase 7: Frontend (React UI)

Step 7.1 — Build the Upload Page

- Drag-and-drop resume upload zone (accept PDF, DOCX).
- Target Job Role input — either a text search or a dropdown of predefined roles from your Neo4j JobRole nodes.
- Analyze button that calls POST /api/analyze with the file + target role.

Step 7.2 — Build the Results Dashboard

Two main sections on the results page:

- Skill Gap Section: Circular match score gauge, color-coded skill table (green = have, orange = partial, red = missing), learning resource links for gaps.
- Job Recommendations Section: Card-based list of top 5–10 jobs with company logo, title, location, match %, key matching skills highlighted, and Apply Now button linking to the original job posting.

Phase 8: Full Data Flow Summary

End-to-End Data Flow

1. [ONCE] Build ontology → Seed Neo4j with ESCO/O*NET skill taxonomy 2. [DAILY] Scraper fetches jobs → NLP extracts skills → Jobs stored in Neo4j 3. [ON UPLOAD] Resume uploaded → Text extracted → Skills parsed via NER 4. [ON ANALYZE] Candidate skills vs. Target Role in Neo4j → Gap report generated 5. [ON ANALYZE] Candidate profile scored against all jobs in Neo4j → Top 10 returned 6. [FRONTEND] Dashboard renders gap report + job cards with apply links

Phase 9: Recommended Build Order

Build in this sequence to validate each layer before adding complexity:

12. Set up Docker Compose (Neo4j + FastAPI + React + Redis running locally)
13. Seed Neo4j with a small manual ontology (10 skills, 3 job roles) to test queries
14. Build resume upload + text extraction (no NLP yet, just raw text)
15. Add spaCy skill extraction from raw resume text
16. Build the Skill Gap Analysis query in Cypher and connect it to the API
17. Build the basic React UI (upload + gap results only)
18. Import ESCO/O*NET into Neo4j to grow the ontology
19. Build the job scraper + NLP enrichment pipeline
20. Add the job recommendation scoring + results UI
21. Add semantic embeddings (Sentence Transformers) for better matching
22. Add user auth (JWT), resume history, and PostgreSQL for user data
23. Deploy to cloud (Railway, Render, or AWS/GCP)

Answers to Your Specific Questions

Is Neo4j the right choice?

Yes, 100%. Neo4j is the industry-standard choice for ontology and knowledge graph problems. The graph model makes skill relationships (subsets, synonyms, prerequisites) natural to express and efficient to query. You can use Neo4j AuraDB Free Tier for development and production at small scale.

Should the ontology be built once or continuously updated?

Build it once from ESCO/O*NET, then auto-extend it when the daily scraper finds new skill terms not already in the graph. Never delete nodes — only add or update relationships. This keeps the graph growing and more accurate over time.

Does the scraper need to run every day?

For a good user experience, yes — daily is ideal. Job postings expire quickly. You can start with weekly scraping during development and move to daily for production. Cache the scraped jobs in Neo4j so you never need to re-scrape the same listing twice (use the job URL as a unique ID).

How are skills linked from jobs to recommendations?

The flow is: Job Description → NER extracts skill names → Skill names matched to Neo4j Skill nodes → (:Job)-[:REQUIRES_SKILL]->(:Skill) edges created. Then at query time, your candidate's skills are matched against these edges to find the best-fitting jobs. The graph structure makes this both fast and explainable.

Your idea is well-thought-out and technically sound.

Follow the phases in order, validate at each step, and you will have a production-ready system.